Task 2: Build the Catalog — Search, Filter, Sort, and Paginate

Phase 2, Task 2 deliverable. Author: Ahmed. Date: 2026-11-12. Branch: `feature/catalog-search-filter-sort`.

Try it: `npm install`, then `npm run start` and open http://127.0.0.1:8080/catalog.html. The site and the mock API run on one origin (`/api/*`). Add `#demo` to the catalog address to get buttons that make the next request fail with 400, 404, 429 or 500, or respond slowly (to see the loading state).

1. What Was Built

Requirement from the brief	Where	Evidence
`fetch` with `async`/`await` inside `try`/`catch`	`src/api/http.js` (`request`), `src/components/catalog-view.js` (`createResultsLoader`)	unit + integration tests
Product cards created from JSON with `createElement` (no `innerHTML`)	`src/components/product-card.js`	`qa/js-rules.js` forbids `innerHTML`; e2e
`map`, `filter`, `reduce` for facet counts and transformations	`src/utils/facets.js` (`countBy`, `facetOptions`, `buildFacets`, `priceRange`), `product-list.js` (`map` to cards)	`tests/unit/facets.test.js`
Debounced search input (300 ms)	`src/utils/debounce.js`, `src/components/filter-bar.js`	e2e: typing "wool" sends 1 request, not 4
Filter panel: category, brand, price range, minimum rating	`catalog.html`, `src/components/filter-bar.js`	e2e journeys 2 and 4
Filters compose with sorting and reset pagination	`applyChange()` in `src/utils/query-string.js`	unit tests; e2e journey 2
Loading, empty and error states with retry	`product-list.js` (placeholders, empty), `status-message.js` (error + retry)	e2e journeys 7 and 8
View synchronised to the URL (shareable, reload-safe, Back/Forward)	`src/state/catalog.js` (`pushState`, `replaceState`, `popstate`)	e2e journey 3
Status-specific error messages (400, 404, 429, 500)	`describeError()` in `src/api/errors.js`	unit tests; e2e journey 8
[v1.1] Cancel superseded requests with `AbortController`	`src/utils/latest.js` (`createLatestRunner`)	unit test; e2e journey 6
Product page (where a 404 really happens)	`product.html`, `src/components/product-detail.js`	e2e journey 9

2. Endpoint Usage

Shopper action	Request(s) sent	Statuses handled
Open the catalog or change any filter, sort or page	`GET /api/products?{filters}&sort&page&limit=6`	200 (results or empty), 400, 404, 429, 500, network
Change search, price or rating (facet counts)	`GET /api/products?q&minPrice&maxPrice&minRating&limit=50&page=n` (every page, in parallel); cached while those four stay the same	errors keep the previous counts
Open a product	`GET /api/products/{id}`	200, 400, 404, 429, 500, network

Contract changes for this task (`api/openapi.json`, checked by `npm run contract`: **50 requests, 0 problems**): `brand` field and filter, `minRating` filter, and `500` documented on both catalog endpoints. The mock also accepts two test-only headers, `X-Mock-Status` and `X-Mock-Delay`, used by the demo panel and the tests.

3. URL Parameters (the view's single source of truth)

Parameter	Control	Example	Notes
`q`	Search box (debounced; replaces the history entry)	`q=wool`	trimmed, max 100 characters
`category`	Category radio group, with counts	`category=electronics`	
`brand`	Brand radio group, with counts	`brand=Northline`	

Parameter	Control	Example	Notes
`minPrice`, `maxPrice`	Price inputs in dollars	`minPrice=3000&maxPrice=8000`	sent in cents, like the API; min > max is refused in the form and swapped if typed in the URL
`minRating`	Minimum rating select	`minRating=4.5`	
`sort`	Sort select	`sort=price_desc`	`price_asc`, `price_desc`, `rating_desc`, `name_asc`
`page`	Pagination	`page=2`	any filter or sort change goes back to page 1; a page past the end goes to the last page

Invalid values in a pasted URL become defaults, so the page never sends a request the API would reject.

4. Loading, Empty and Error States

State	What the shopper sees	Action	How to see it
Loading	6 placeholder cards, list `aria-busy="true"`, status "Loading products…"	—	`#demo` → "Slow response (3 s)"
Empty	"No products match your filters"	"Clear all filters"	search for `zzz`
400	"Some filters are not valid" + the field names	"Clear all filters"	`#demo` → 400
404	"Not found" (catalog) / "Product not found" (product page)	"Clear all filters" / "Browse all products"	`#demo` → 404; `product.html?id=nope`
429	"Too many requests… Wait N seconds"	"Try again in N s" counts down, then enables	`#demo` → 429
500	"Something went wrong on our side"	"Try again"	`#demo` → 500
No connection	"Could not reach the store"	"Try again"	stop the server

Errors are announced (`role="alert"`); the result count is a polite live region (`role="status"`).

5. No Stale Results (debounce + AbortController)

- Typing is debounced for 300 ms; pressing Enter searches at once.
- Each new view runs through `createLatestRunner()`: it **aborts** the previous `fetch` with its `AbortController` and, as a second guard, **drops** any answer that arrives after a newer request started.
- Proven by e2e journey 6: the electronics request is held for 1.5 s, apparel is chosen meanwhile; the apparel results stay on screen after the slow answer is due and the browser reports the old request cancelled. With the guard disabled the same journey fails (checked on 2026-11-12).

6. Results Match the Fixtures

`tests/fixtures/catalog-cases.json` lists 12 queries with the exact product ids, totals and page counts expected from the mock catalog (12 products, 5 brands). They were computed separately from the mock server and are checked three times: by the integration tests (real `api/` modules against the mock), by the e2e journeys (what the page shows), and partly by `npm run contract`.

Fixture case	Expected ids
default view, page 1 / page 2	6 + 6 products in catalogue order
category `electronics`	headphones-pro, smartwatch, bluetooth-speaker, usb-c-hub
`electronics` + brand `Pulse`	smartwatch, usb-c-hub
brand `Northline`, price high to low	rain-shell, leather-backpack, merino-tee, card-wallet
price $30–$80, price low to high	ceramic-mug, usb-c-hub, merino-tee, bluetooth-speaker, desk-lamp, linen-throw
rating ≥ 4.6, top rated	wool-socks, headphones-pro, leather-backpack, smartwatch, linen-throw
search `wool`	wool-socks
apparel, rating ≥ 4.2, max $120, price low to high, 2 per page, page 2	rain-shell
search `zzz`	(none: empty state)

7. QA Matrix

Journey (`npm run e2e`)	Chrome desktop 1280	Chrome phone 375	Firefox	Safari / WebKit
1. Default view and pagination match the fixtures; page change moves focus to the results	Pass	Pass	Not run	Not run
2. Category, brand and sort compose; a filter change resets the page	Pass	Pass	Not run	Not run
3. View survives reload and Back/Forward	Pass	Pass	Not run	Not run
4. Price range (dollars → cents) and rating; invalid range refused	Pass	Pass	Not run	Not run
5. Search is debounced (one request per burst of typing)	Pass	Pass	Not run	Not run
6. Slow superseded request is aborted and never overwrites the newer view	Pass	Pass	Not run	Not run
7. Empty state and "Clear all filters"	Pass	Pass	Not run	Not run
8. Loading, 400, 404, 429 (countdown) and 500 (retry) states	Pass	n/a (desktop only)	Not run	Not run
9. Product page opens from a card; unknown id shows "Product not found"	Pass	Pass	Not run	Not run
Phase 1 journeys (home, filters, deals, newsletter, keyboard, menu, comparison, 404)	8 of 8 pass	9 of 9 pass	Not run	Not run

Total: **34 of 34 journey runs pass** in Chromium. Firefox and WebKit are not installed in the build environment (`npx playwright install firefox webkit`, then `E2E_BROWSERS=chromium,firefox,webkit npm run e2e`).

Other check	Result
`npm run lint` / `npm run format:check`	0 problems / all formatted
Unit tests (`tests/unit`, 71) and integration tests against the mock and the Netlify function (`tests/integration`, 22)	93 of 93 pass (Node test runner through a Vitest-compatible shim; see deviation 3)
`npm run contract`	50 requests, 0 problems
`npm run build`	OK: 38 modules reachable from the pages, all references resolve
`node qa/js-rules.js` (ES modules only, no inline scripts, no globals, every function commented and ≤ 40 lines)	0 failures (51 files, 140 functions)
`node qa/preflight.js` (accessibility and SEO rules, all 5 pages)	269 of 269
`node qa/headers.js` (Netlify headers + CSP; catalog loads from `/api` with no violations)	0 failures
Phase 1 behaviour, layout, keyboard, device-matrix and contrast checks	all pass

QA found and fixed four bugs: the "Clear all filters" button was invisible (an outline style meant for dark backgrounds), "Shop" was a tap target narrower than 44 px, long product names overflowed cards at 200% text size, and the product page's error box skipped a heading level (h1 → h3).

8. Deviations List

#	Deviation	Reason
1	The "live API" is the project's mock server, served on `/api` by `npm run start`, Vite, and a Netlify Function in production	No external API was provided; one origin also keeps the CSP (`default-src 'self'`) strict
2	Facet counts come from fetching every matching product (at most 50 per page, pages in parallel) and counting with `reduce`	The brief asks for client-side counts; with 12 products this is one request. A large catalog would need server-side facets
3	Unit and integration tests were run with Node's test runner through a small Vitest-compatible shim; MSW not used	Vitest and MSW could not be installed in the build environment. The integration tests use the real mock server instead of MSW; `npm test` runs them with real Vitest
4	Error states are demonstrated with test-only request headers (`X-Mock-Status`, `X-Mock-Delay`) via the `#demo` panel	Lets reviewers see 400/404/429/500 on the live site without breaking normal browsing
5	Category and brand are single-choice	Matches the API (one value per parameter)

#	Deviation	Reason
6	Firefox, Safari and real devices not tested	Not available in the build environment
7	Netlify Drop cannot run functions: the catalog needs a Git-connected Netlify site (or `netlify deploy`) to have `/api`	Static hosting alone has no API